PAYPAL AI HACKATHON · AGENTIC COMMERCE TRACK

PropLock AI
Feature 1: Multimodal Listing Audit & Commute-Truth Agent

Technical review, issues and fixes, real-world broker analysis, and target architecture

9 / 9	8	9	4
existing tests pass	major issues confirmed by running the code	medium issues	checks in the target design (A–D)

Verdict. The skeleton is solid (FastAPI → LangGraph fan-out → 3 checks → decision engine → PayPal Catalog), but the trust logic has gaps that let a dishonest broker earn a **VERIFIED** badge, and failures are hidden behind fallbacks. This document lists each gap with the buggy code, the evidence, and a concrete fix.

Codebase reviewed: `D:\hackathon\paypal\backend`
Reference: *PropLock AI Product Blueprint* (5 pages) and the Feature 1 flow diagram
Review date: 22 October 2026

Contents

1. Executive summary
2. What Feature 1 does: checks A–D
3. Current implementation
4. Major issues (M1–M8)
5. Medium and minor issues
6. How rental brokers work in real life
7. Broker scams and how Feature 1 catches them
8. Fake photo defenses
9. Commission vs rent markup
10. Target architecture (all fixes applied)
11. Integrating the broker workflow
12. Existing solutions in the market
13. Build plan for the demo
14. Appendix: probe results and sources

1. Executive summary

Feature 1 is PropLock's **trust layer**. Before a broker's rental ad goes live, it checks the ad against three sources of truth, the owner's verified record, real map routes and state tenancy rules, and only then mints the listing as a PayPal Catalog product with a *Verified Commute & Refund Protected* badge. Features 2–4 (qualification, escrow hold, invoicing) all rely on that verified record.

Area	Status
Architecture and code structure	GOOD Clean separation of API, agents, services and DB; compiled LangGraph StateGraph with parallel fan-out.
Test suite	PASSES 9/9, but tests rely on filename heuristics and fallbacks, so they don't cover the bypasses below.
Trust guarantees	BROKEN Rent/furnishing never compared with the owner (M1); lies in ad text skip the commute check (M2); audit hash doesn't lock broker photos (M4); unauthenticated ground-truth writes (M6).
Failure handling	FAIL-OPEN PayPal failure returns a fake product ID (M5); DB errors swallowed and reports fabricated (M7); commute fallback invents evidence (M3).
Reproducibility	BROKEN `requirements.txt` lacks `sqlalchemy` and `langgraph` (M8).

Recommended fix order: M8 → M1 → M2 → M5 → M3 → M4 → M7 → M6. The first four make the demo honest; the rest make it robust.

2. What Feature 1 does: checks A–D

Every listing goes through four independent checks, run in parallel. A, B and C exist in the code today; D is the new check proposed in this review.

#	Check	Question it answers	Compared against	Source file
A	Photo inspector	Are these real photos of *this* flat, and do they match the claims?	Owner baseline photos, the web, capture GPS	`agents/photo_inspector.py`

B	Commute truth	Is "5 min to metro" true?	Peak-hour route from the owner's verified coordinates	`agents/distance_calculator.py`
C	Rental compliance	Is the deposit legal in this state?	State tenancy rules (TN, Karnataka, ...)	`agents/law_checker.py`
D	Ground truth NEW	Do rent, deposit, furnishing match what the owner agreed?	Owner record / broker mandate	`agents/ground_truth_checker.py`

Check A: photo inspector

Sub-check	Example it catches	In code today?
Watermark detection	Photo copied from NoBroker with its logo	✅ Gemini, or filename guess without a key
Furnishing match	Claims "Fully Furnished", photo shows an empty room	✅
Same room as baseline	Photo of a different flat from the web	❌ hash computed, then discarded
Taken at the flat (GPS)	Photo not taken at the property	❌ new
Reverse image search	Photo exists elsewhere online	❌ new
Duplicate across properties	Same photo used for two different flats	❌ new

Check B: commute truth

```
Broker says:  "5 min walk to Indiranagar Metro"
Maps says:    22 min walk at 08:30 on a weekday
Allowed:      max(5 × 1.2, 5 + 5) = 10 min
22 > 10  →  FAIL      (no route data → UNVERIFIABLE, never an invented number)
```

The origin is the owner's verified coordinates, which the broker cannot change; the destination and minutes come from the broker's claims.

Check C: rental compliance

```
Chennai flat: rent ₹40,000, deposit ₹2,40,000  →  6 months
Tamil Nadu rule: max 3 months                  →  FAIL, with legal citation
```

Check D: ground truth (new)

```
Owner mandate:   rent ₹32k–₹35k, deposit ≤ ₹70k, Unfurnished, brokerage ≤ 1 month
Broker listing:  rent ₹40k,       deposit ₹1.4L,  Fully Furnished
                 ❌ over range     ❌ over owner    ❌ owner says Unfurnished
```

No AI or external API is needed: it compares numbers already in your database. It is the cheapest check and catches the most common scam.

How the results combine

A	B	C	D	Outcome
PASS	PASS	PASS	PASS	**VERIFIED** → PayPal product + badge
PASS	FAIL	PASS	PASS	**NEEDS_FIX** → evidence and suggested fix sent to broker
PASS	UNVERIFIABLE	PASS	PASS	**IN_REVIEW** → a manager decides

3. Current implementation

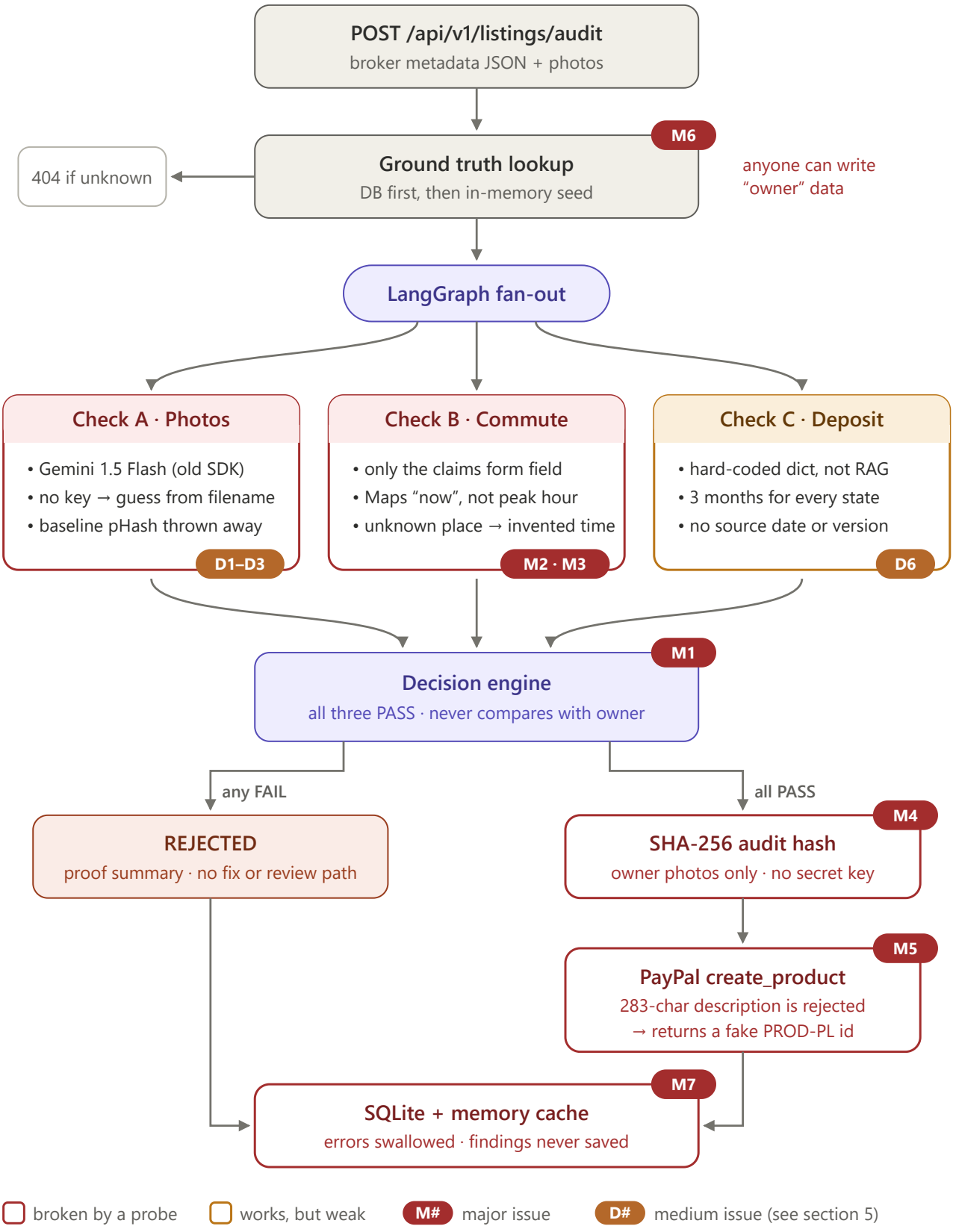


Figure 1. Current flow. Red steps were broken by at least one probe.

Component	File	What it really does
API	`api/v1/listings.py`	`POST /listings/audit`, `GET /listings/audit/{id}`, `GET /listings/{key}`
Ground truth	`db/ground_truth_store.py`	Reads the DB first, then an in-memory seed. `POST /ground-truth` writes only to memory.
Orchestrator	`agents/orchestrator.py`	Compiled LangGraph: START → 3 parallel nodes → decision_engine → END
Decision	`agents/decision_engine.py`	Requires all three PASS, builds the audit hash, mints the PayPal product
PayPal	`services/paypal_catalog.py`	OAuth on every call, `create_product`; any failure returns a fabricated ID

Blueprint diagram vs code

Diagram step	Status in code
A: Perceptual hash + CLIP	⚠️ pHash computed, result thrown away; no CLIP
A: Compare with reference	❌ not used in the verdict
A: Gemini Vision check	⚠️ `gemini-1.5-flash` via deprecated SDK (blueprint says 2.5 Flash)
B: Extract address + claim	❌ only the structured `commute_claims` field is read
B: Mock DB or Maps API	⚠️ uses "now" not peak hour; mock invents numbers
B: Compare claim vs route	✅ `max(claimed×1.2, claimed+5)`
C: RAG finds legal rules	❌ hard-coded dict; markdown acts never read
C: Python rule check	✅
C: Cite source + date	⚠️ source cited, no date or version
Decision engine: rules + evidence thresholds	⚠️ rules yes; `confidence_score` never used
Audit dashboard	❌ no frontend in the repo
No → "Fix or manual review"	❌ only VERIFIED / REJECTED
Yes → PayPal Catalog product	⚠️ happy path works, but fails over to a fake ID

4. Major issues (M1–M8)

Each issue was reproduced on a scratch copy of the backend with a fresh database and no API keys. Probe IDs (P1–P9) refer to the appendix.

M1 Broker claims are never compared with the owner's data

Where: `agents/decision_engine.py` lines 32–36. **Evidence:** P1. A broker charging ₹55k (owner agreed ₹35k) and advertising "Fully Furnished" for an unfurnished flat is **VERIFIED**, and the inflated price is minted into PayPal.

BUGGY CODE

```
all_passed = (
    image_audit.verdict == "PASS" and
    commute_audit.verdict == "PASS" and
    compliance_audit.verdict == "PASS"
)   # claimed_rent / deposit / furnishing never checked against owner_truth
```

FIX: ADD CHECK D

```
FURNISH_RANK = {"Unfurnished": 0, "Semi-Furnished": 1, "Fully Furnished": 2}

def check_ground_truth(sub, mandate, owner) → list[str]:
    issues = []
    if not (mandate.min_rent ≤ sub.claimed_rent ≤ mandate.max_rent):
        issues.append(f"Rent {sub.claimed_rent} outside owner range {mandate.min_rent}–
{mandate.max_rent}")
    if sub.claimed_deposit > mandate.max_deposit:
        issues.append(f"Deposit {sub.claimed_deposit} > owner-agreed {mandate.max_deposit}")
    if FURNISH_RANK[sub.claimed_furnishing] > FURNISH_RANK[owner.furnishing_status]:
        issues.append(f"Claims {sub.claimed_furnishing}, owner registered
{owner.furnishing_status}")
    if sub.brokerage_fee > mandate.max_brokerage_months * sub.claimed_rent:
        issues.append("Brokerage above agreed cap")
    return issues   # non-empty → FAIL
```

M2 Lies in the title or description skip the commute check

Where: `agents/distance_calculator.py` lines 18–25. **Evidence:** P2. A title of "2 MIN WALK TO METRO" with an empty `commute_claims` list is **VERIFIED** with `claims_evaluated = 0`. Tenants read the title, not a hidden form field.

BUGGY CODE

```
if not claims:
    return CommuteAuditResult(verdict="PASS", claims_evaluated=0, ...)
```

FIX: EXTRACT CLAIMS FROM THE AD TEXT FIRST

```
class ExtractedClaims(BaseModel):
    commute: list[CommuteClaim]

resp = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=f"Extract every commute claim from this ad:\n{sub.title}\n{sub.description}",
    config={"response_mime_type": "application/json", "response_schema": ExtractedClaims},
)
claims = sub.commute_claims + resp.parsed.commute
```

M3 The fallback commute check invents evidence

Where: `services/maps_service.py` lines 70 and 75–77. **Evidence:** P3, P4. For any destination not in the hard-coded table, the "actual" time is `claimed + 12`, which always exceeds the tolerance. An honest 30-minute claim is reported as "actual 42 min, LIE DETECTED". The two-way substring match also lets destination `"a"` match "tech park".

BUGGY CODE

```
if p_id == property_id and (kw in dest_clean or dest_clean in kw) and m == mode:
    ...
if not matched:
    actual_minutes = max(claimed_minutes + 12, 18)   # always above tolerance
```

FIX: SAY "CAN'T VERIFY" AND MATCH EXACTLY

```
if (p_id, kw, m) == (property_id, dest_clean, mode):
    ...
if not matched:
    return CommuteFinding( ... , actual_peak_minutes=None, status="UNVERIFIABLE",
                           summary="No route data; sent to manual review.")
```

M4 The audit hash doesn't lock the broker's photos

Purpose of the feature: fingerprint the verified listing so that any later change (rent, photos, claims) is detected before checkout. This is what stops bait-and-switch after the badge is issued.

Where: `agents/orchestrator.py` lines 52–57, `services/paypal_catalog.py` line 37. **Evidence:** P6. Two submissions with different photos produce the **same** hash.

BUGGY CODE

```
photo_hashes = list(owner.baseline_photo_hashes)        # OWNER photos, not the broker's
commute_claim_str = f"{sub.commute_claims[0] ... }"     # only the FIRST claim
return hashlib.sha256(payload.encode("utf-8")).hexdigest()   # unsigned: anyone can recompute
```

FIX: FINGERPRINT EVERYTHING CHECKED, SIGNED WITH A SERVER SECRET

```
payload = json.dumps({
    "submission": sub.model_dump(),                                    # all claims, rent,
deposit
    "photo_phashes": sorted(compute_phash(b) for _, b in photos),      # BROKER photos
    "owner": owner.model_dump(), "audit_id": audit_id, "ruleset": "v3",
}, sort_keys=True, separators=(",", ":"))
signature = hmac.new(settings.AUDIT_SECRET.encode(), payload.encode(),
hashlib.sha256).hexdigest()
```

M5 PayPal failures quietly become fake success

Where: `services/paypal_catalog.py` lines 91, 100, 114–121. **Evidence:** P5. The description is 283 characters; Catalog Products v1 allows 256, so a real sandbox call is rejected and the code returns an invented `PROD-PL-…` ID. `category: "REAL_ESTATE"` should also be checked against PayPal's category list. Catalog products can be PATCHed, so immutability must come from your signed audit, not from PayPal.

BUGGY CODE

```
description = f"PropLock Verified. {json.dumps(pricing_metadata)}"   # 283 chars
...
if resp.status_code in (200, 201):
    return resp.json().get("id")
return f"PROD-PL-{property_id}-{broker_id}-{short_hash}"             # FAKE id
```

FIX: SHORT DESCRIPTION, OWN PRODUCT ID, RAISE ON FAILURE

```
body = {
    "id": f"PL-{submission_id}",                                    # deterministic, retry-
safe
    "name": f"PropLock - {title[:100]}",
    "description": f"PropLock audit {audit_id} sig {signature[:16]}", # < 256 chars
    "type": "SERVICE",
    "home_url": f"https://proplock.ai/audits/{audit_id}",
}
resp = await client.post(url, json=body, headers=headers)
if resp.status_code not in (200, 201):
    logger.error("PayPal %s debug_id=%s %s", resp.status_code,
                 resp.headers.get("PayPal-Debug-Id"), resp.text)
    raise PayPalMintError(resp.text)        # audit stays PENDING_MINT and is retried
```

M6 Anyone can write the "source of truth"

Where: `api/v1/ground_truth.py` lines 18–21; `api/v1/listings.py` lines 76–85. **Evidence:** P7. A self-registered property with made-up owner data is **VERIFIED**. No endpoint has authentication; unknown brokers are auto-created with a fake licence; ground-truth writes go only to memory.

BUGGY CODE

```
@router.post("")                                          # no auth
async def register_owner_ground_truth(payload: OwnerGroundTruth):
    return ground_truth_db.register_property(payload)   # in-memory only
...
broker = Broker(broker_id=submission.broker_id, license_number=f"LIC-{submission.broker_id}",
... )
```

FIX: ROLES, PERSISTED WRITES, BROKER IDENTITY FROM THE TOKEN

```
@router.post("", dependencies=[Depends(require_role("MANAGER"))])
async def register_owner_ground_truth(payload: OwnerGroundTruth, db: Session =
Depends(get_db)):
    db.merge(Property(**to_row(payload))); db.commit()

@router.post("/audit")
async def audit_listing( ... , broker: Broker = Depends(current_broker)):
    submission.broker_id = broker.broker_id             # never trust the request body
```

M7 Persistence fails silently and stored reports are fabricated

Where: `api/v1/listings.py` lines 111–116 and 151–158. A DB failure is swallowed after PayPal has already created the product (an orphan). Photos, claims and findings are never written. Reading a report back returns invented values. The persistence test writes into the real dev database.

BUGGY CODE

```
except Exception:
    pass                                  # audit lost, PayPal product orphaned
...
compliance_audit=ComplianceAuditResult(
    state="Verified Region", demanded_deposit_months=2.0,      # INVENTED
    statutory_max_deposit_months=3.0, statute_citation="Statutory Compliance")
```

FIX: STORE THE REAL REPORT, SAVE BEFORE MINTING, LET ERRORS RAISE

```
report_json = Column(Text, nullable=False)                     # models.py

audit_rec = ListingAudit( ... , overall_verdict="PENDING_MINT",
                         report_json=report.model_dump_json())
db.add_all([sub_rec, audit_rec]); db.commit()                   # one transaction, BEFORE
PayPal

return AuditReport.model_validate_json(db_audit.report_json)   # read back exactly what was
stored
```

M8 A fresh install crashes

Where: `requirements.txt`. `sqlalchemy` and `langgraph` are imported but not listed, so a judge cloning the repo gets `ModuleNotFoundError`. The code also uses the deprecated `google-generativeai` SDK.

FIX

```
sqlalchemy≥2.0
langgraph≥0.2
google-genai≥1.0
# remove google-generativeai
```

5. Medium and minor issues

#	Issue	Where	Fix
D1	Photo check falls back to the **filename**; silent on Gemini errors	`gemini_vision.py:114` `any(w in fn_lower …)`	No Gemini → `NEEDS_REVIEW`; heuristic only when `DEMO_MODE=true`
D2	Baseline pHash result thrown away; seeded hashes are made up	`gemini_vision.py:54–63`	Use it; add embedding similarity and cross-property duplicate check
D3	Missing Gemini fields default to pass; free-text JSON parsing; confidence ignored	`gemini_vision.py:97`	`response_schema`, default to fail, confidence threshold
D4	Sync Gemini and SQLAlchemy calls block the event loop	services, API handlers	`client.aio`, `asyncio.gather`, `run_in_threadpool`
D5	Commute uses "now", not peak hour; destination not city-scoped; element status unchecked	`maps_service.py:54`	Next weekday 08:30 IST epoch; geocode "{dest}, {city}"; Routes API
D6	Legal "RAG" is a hard-coded dict (3.0 months everywhere); some citations look inaccurate	`rag_engine.py:19–32`	Versioned rules file with section, source URL, effective date; verify against official texts
D7	No input validation: rent 0 + deposit 999999 passes (P9); zero photos pass at orchestrator level (P8)	`broker_submission.py:15–16`	`Field(gt=0)`; 1–20 images, size and type limits
D8	One node exception fails the whole graph with a 500	`orchestrator.py`	Per-check timeout; error → `UNVERIFIABLE`
D9	Public listing shows the badge for a rejected submission if the broker has another verified audit	`listings.py:215–221`	Resolve the badge from that submission's own audit

Minor

- `allow_origins=["*"]` with `allow_credentials=True`: restrict CORS to the frontend origin.
- `DATABASE_URL` read via `os.getenv`, so the `.env` value is ignored; relative SQLite path depends on the working directory.
- `init_db()` swallows exceptions; `datetime.utcnow()` is deprecated.
- In-memory and DB seeds disagree; `get_property` invents default hashes and URLs.
- `_audit_reports_cache` is an unbounded global, not shared across workers.
- Tests mutate the class-level `KNOWN_COMMUTE_MATRIX`; no API-level or mocked-HTTP tests.
- Karnataka 3–6 month advisory never reaches `proof_summary`, so tenants don't see the warning.
- `proplock_truth.db` and `.pytest_cache/` belong in `.gitignore`.

6. How rental brokers work in real life

Based on common practice in Indian metros (Bengaluru, Chennai, Mumbai, Delhi). Fees and customs vary by city and over time.

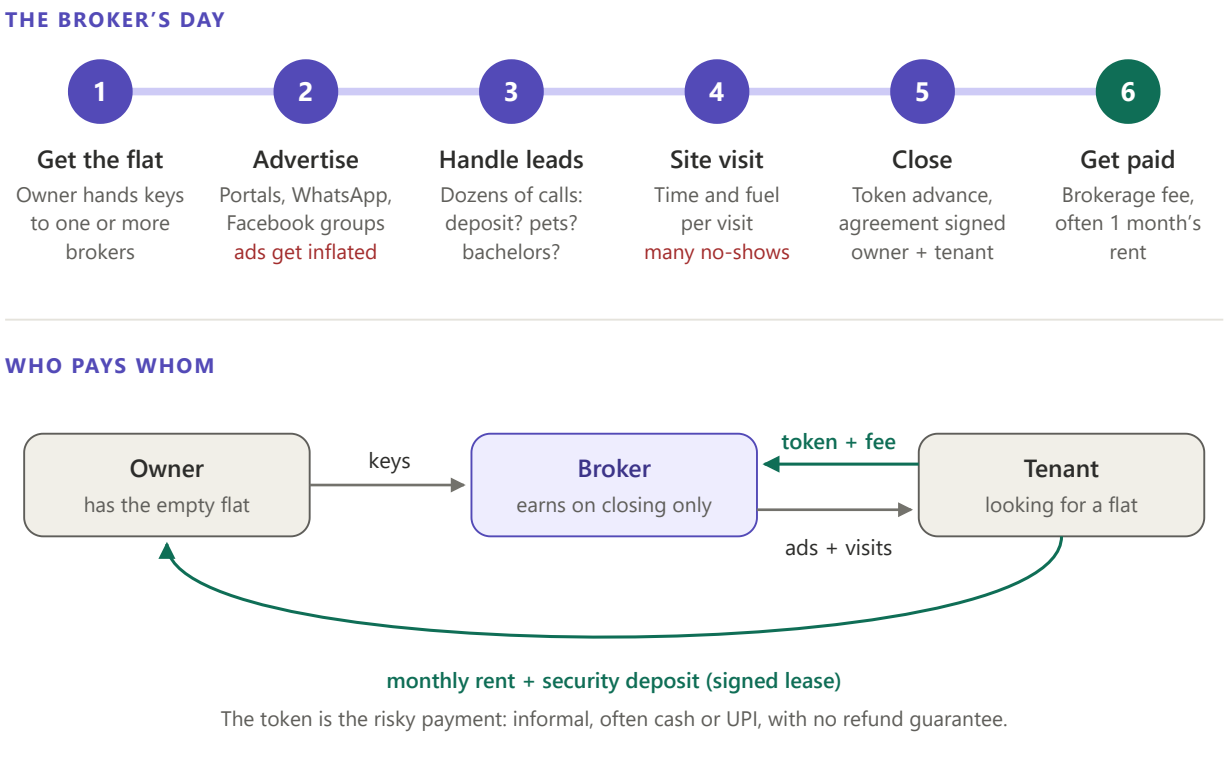


Figure 2. Typical rental broker workflow.

1. **Getting the flat.** A busy or non-resident owner hands keys to one or more brokers, or to an agency such as "XYZ Properties" that uses sub-brokers.
2. **Advertising.** Ads on portals, WhatsApp and Facebook. Several brokers compete for the same flat, so ads get exaggerated.
3. **Handling leads.** Dozens of calls a day about availability, deposit, pets, bachelors.
4. **Site visits.** Time and fuel spent per visit; many tenants don't turn up.
5. **Closing.** Token advance, rent agreement between owner and tenant, deposit to the owner.
6. **Getting paid.** A separate brokerage fee, commonly one month's rent (sometimes 15 days), usually from the tenant and sometimes from the owner as well.

7. Broker scams and how Feature 1 catches them

#	Scam	How it works	Caught by
1	Fake or stolen photos	Photos of a nicer flat copied from another portal or brochure	Check A: live capture, same-room check, reverse image search
2	Bait listing	Cheap ad for a flat that doesn't exist or is rented; "that one's gone, but I have another..."	Mandate (only assigned flats can be listed); Feature 3 void on fake flat
3	Commute lies	"5 min to metro" measured as straight-line distance	Check B

4	Furnishing lies	"Fully furnished" is a bed and a fan	Check A furnishing + Check D
5	Rent markup	Lists ₹40k, owner gets ₹35k, broker keeps the difference	Check D (owner's rent range)
6	Inflated deposit	Asks 10 months, keeps part or delays passing it on	Check C (legal cap) + Check D (owner deposit)
7	Token advance scam	"Pay ₹10k to hold the flat", then disappears	Feature 3: PayPal authorization hold, auto-voided
8	Visit or registration fee	Charges before showing any flat	Feature 3: hold becomes payment only by agreed rules
9	Fake broker or owner	Someone unconnected collects advances from several tenants	Authentication + manager-only ground truth (M6)
10	Hidden charges	Extra fees revealed only at signing	`brokerage_fee` field + "total move-in cost" card
11	Double-booking	Takes tokens from several tenants for one flat	Feature 2/3: one active escrow lock per property

Scams on the owner, and on the broker

Against the owner	Against the broker
Unauthorised subletting Rent skimming Pushing unvetted tenants	Tenant no-shows (Feature 3: ₹500 hold captured on verified no-show) Tenant bypasses broker to the owner (Feature 4: token tied to that broker's audited listing) Owner pulls out after a tenant is found

Pitch line. Today every party protects itself by distrusting everyone else. PropLock replaces "trust the person" with "trust the evidence": Feature 1 proves the listing is true, Feature 3 proves who showed up, and PayPal holds the money until the evidence decides.

8. Fake photo defenses

The audit hash only detects changes *after* verification. If a broker uploads a web photo *at* submission, the hash simply locks in the fake. Catching it is Check A's job, and today Check A misses it: no watermark, a harmless filename, a furnished-looking room, and the baseline comparison is discarded.

Layer	How	Stops a web photo?
0. Show only owner photos	The public listing displays XYZ Properties' baseline photos; the broker writes the text only	✅ fully; already half-built
1. Live in-app capture	Camera page records GPS and time; reject if > 100 m from the property or older than 30 days	✅ strong
2. Same room as baseline	Gemini compares windows, doors, flooring, room shape; ignores furniture	✅ good
3. Reverse image search	Google Cloud Vision Web Detection or TinEye API	✅ if the photo is online
4. Own pHash database	Same photo already used for another property	✅ reuse across properties

```
def check_live_capture(meta, owner):
    dist = haversine_m(meta.lat, meta.lng, owner.coordinates.latitude, owner.coordinates.longitude)
    if dist > 100:
        return "FAIL", f"Photo taken {dist:.0f} m from the property"
    if meta.captured_at < now() - timedelta(days=30):
        return "FAIL", "Photo older than 30 days"
    return "PASS", "Captured on-site"

SAME_ROOM_PROMPT = """Image 1 is the owner's verified photo of the empty flat.
Image 2 is the broker's listing photo. Ignore furniture. Compare windows, doors,
flooring, walls, ceiling and room shape. Is this the SAME physical room?
Return JSON: {"same_room": bool, "confidence": float, "mismatched_features": [str]}"""
```

For the demo: show only owner photos, plus the Gemini same-room check on any extra broker photos. Those two close the hole with the least work.

9. Commission vs rent markup

Brokers do need to earn commission, but in the normal Indian rental market commission is **not** added into the rent. The rent agreement is signed by owner and tenant, so a hidden markup surfaces at signing or pushes payments outside the lease.

Model	Who pays	Rent markup OK?
Normal brokerage (separate fee)	One-time fee	❌ No. Rent goes to the owner.
Asking price vs final price	Nobody extra	✅ Within the owner's range

Rent-to-rent / master lease / co-living	Built into the rent	✅ Only with the owner's written consent; operator is the landlord on the lease
Hidden markup	Hidden in rent	❌ This is the bait-and-switch PropLock exists to stop

So Check D compares the listing against an **owner-approved range**, and commission is declared in its own `brokerage_fee` field. The tenant then sees the true move-in total: `rent + deposit + brokerage_fee`. Brokerage norms are market practice, not law; present them as agency policy in the app.

Commute: who is the referee?

Commute claims are checked with the Maps API, not against the owner. The owner supplies only the **origin** (verified coordinates); the broker supplies the claim; Maps is the referee. The weak spot is the origin itself: capture GPS at key handover, or geocode the registered address and reject a pin more than ~200 m away.

10. Target architecture (all fixes applied)

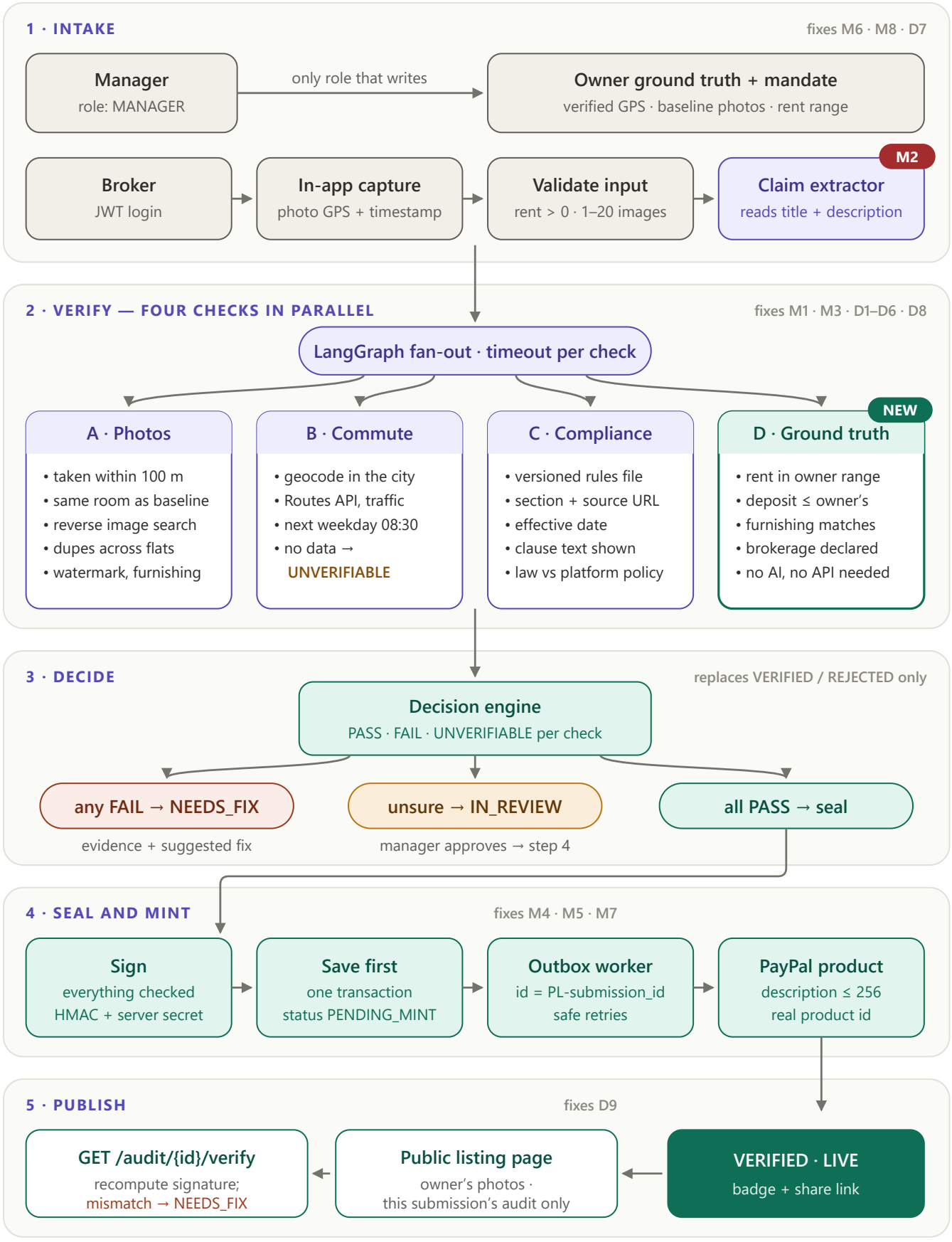


Figure 3. Feature 1 with every fix applied. Labels show the issue each part fixes.

Key design decisions

1. **Three-state results.** Every check returns PASS, FAIL or UNVERIFIABLE with evidence and its source. A missing key or timeout never becomes PASS, and never an invented "LIE".
2. **Check D.** Deterministic comparison with the owner mandate; cheapest check, most common scam.
3. **Claim extraction first.** The LLM reads the ad; deterministic code judges it.
4. **Human review** via LangGraph `interrupt()` with a checkpointer, implementing the blueprint's "Fix or manual review" branch.
5. **Signed, versioned audit.** Canonical JSON → HMAC; `GET /audit/{id}` returns exactly what was signed and `/verify` recomputes it.
6. **Outbox for PayPal.** Commit first, then mint with a deterministic product ID and `PayPal-Request-Id`; retries are safe and nothing is orphaned.
7. **Rules as data.** Caps in a versioned file with `effective_date` and `source_url`; retrieval only shows the clause text.
8. **Roles.** MANAGER writes ground truth and mandates, BROKER submits listings, PUBLIC reads verified listings.

11. Integrating the broker workflow

Feature 1 should follow the broker's real day (get the keys, visit the flat, post the ad, handle leads, mark it rented) instead of being a one-shot "submit and judge" endpoint.

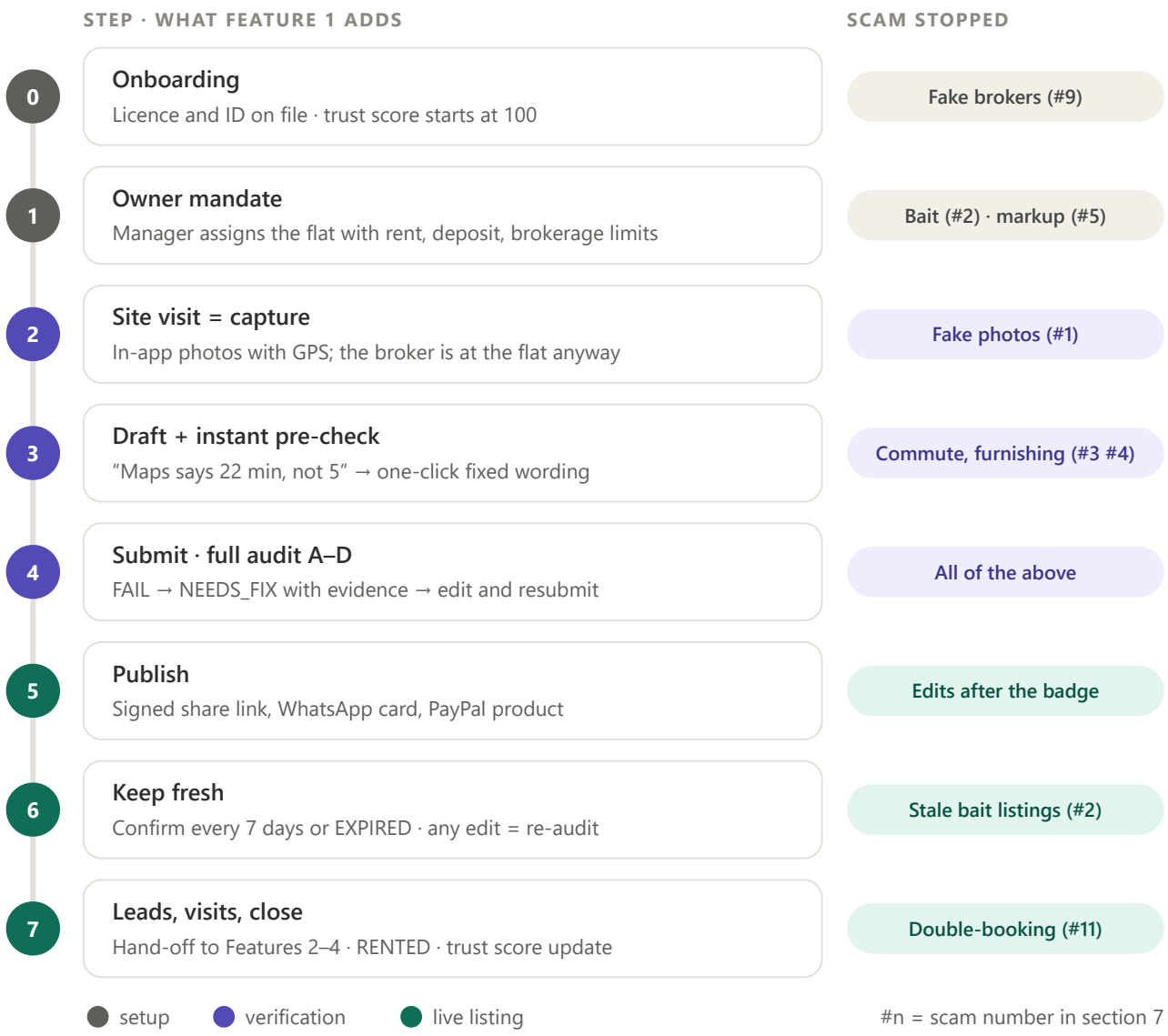


Figure 4. Broker journey with verification built in.

The mandate table

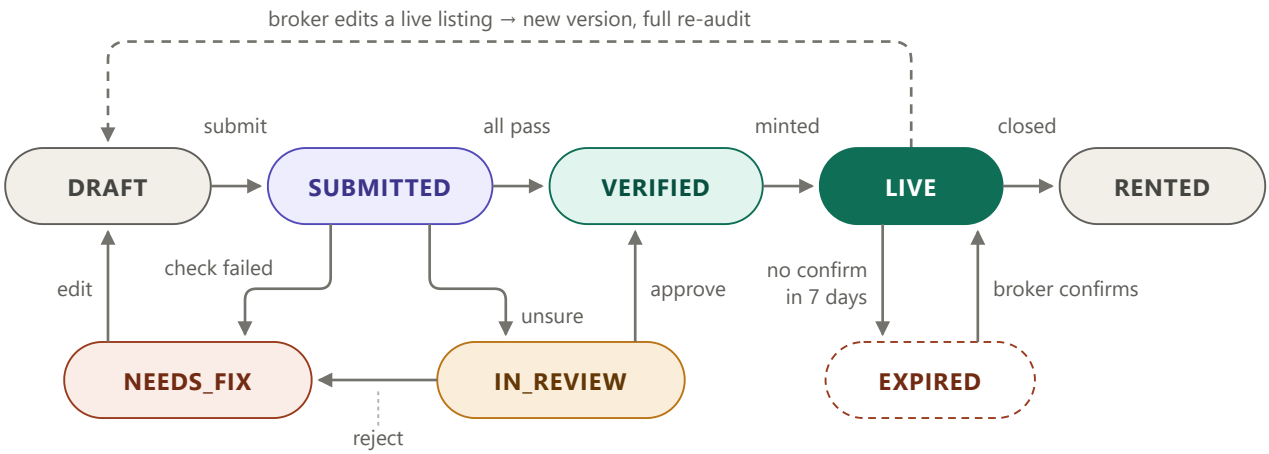
```
class Mandate(Base):
    __tablename__ = "mandates"
    mandate_id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
    property_id = Column(String(50), ForeignKey("properties.property_id"), nullable=False)
    broker_id = Column(String(50), ForeignKey("brokers.broker_id"), nullable=False)
    min_rent = Column(Numeric(12, 2), nullable=False)
    max_rent = Column(Numeric(12, 2), nullable=False)
    max_deposit = Column(Numeric(12, 2), nullable=False)
    max_brokerage_months = Column(Numeric(4, 2), default=1.0)
    valid_until = Column(DateTime, nullable=False)
    status = Column(String(20), default="ACTIVE")    # ACTIVE | REVOKED | EXPIRED

# first thing in audit_listing
mandate = db.query(Mandate).filter_by(property_id=submission.property_id,
                                      broker_id=broker.broker_id, status="ACTIVE") \
            .filter(Mandate.valid_until > now()).first()
if not mandate:
    raise HTTPException(403, "You are not authorised to list this property.")
```

The pre-check: help honest brokers

```
@router.post("/listings/precheck")
async def precheck(draft: BrokerSubmissionMetadata, broker=Depends(current_broker)):
    claims = await extract_claims(draft)
    suggestions = []
    for c in claims.commute:
        f = await maps_service.calculate_commute( ... )
        if f.tolerance_exceeded:
            suggestions.append({
                "problem": f"'{c.claimed_time_minutes} min to {c.destination}': "
                           f"Maps says {f.actual_peak_minutes} min at 8:30 AM",
                "suggested_text": f"{f.actual_peak_minutes} min {c.mode} to {c.destination}",
            })
    return {"ok": not suggestions, "suggestions": suggestions}   # nothing saved or minted
```

Listing lifecycle



Solid pills are the happy path. Every edit creates a new signed version, so old terms stay on record for disputes.

Figure 5. Listing states (replaces VERIFIED / REJECTED only).

Trust score and endpoints

Trust event	Points
Verified on first try	+2
Needs fix	−1
Photo found online	−15
Listing expired while stale	−3
Visit completed (Feature 3)	+1
Broker no-show (Feature 3)	−10

Endpoint	Role	Purpose
`POST /mandates`	Manager	Assign broker to property with limits
`POST /listings/drafts`	Broker	Create a draft
`POST /listings/drafts/{id}/photos`	Broker	Upload GPS-checked captures
`POST /listings/precheck`	Broker	Instant feedback; nothing saved
`POST /listings/{id}/submit`	Broker	Full audit
`POST /listings/{id}/confirm-availability`	Broker	Keep the listing live
`POST /reviews/{audit_id}`	Manager	Approve or reject IN_REVIEW
`GET /brokers/me/dashboard`	Broker	Listings, fixes needed, trust score

12. Existing solutions in the market

Each part of Feature 1 exists somewhere, but no product found combines photo, commute and deposit verification into one verdict that gates a payment. That combination is PropLock's differentiator.

Feature 1 part	Real-world solution	How it compares
Photo audit	**Restb.ai Photo Compliance** (Heartland MLS, MLS Now, CVR MLS): watermarks, logos, added text, people, contact info, duplicates, virtual staging	Enforces MLS photo rules; does not check the photos are of *this* property
	Reverse image search (TinEye API, Google Lens / Cloud Vision Web Detection)	Usable as Layer 3 of the photo check
Fake listing detection	**Onerent Scamlord.ai** (2020); Google patent US9990639 (ML fraud probability); Zillow report tool	Look at scammer patterns, not claim truth
India verified badges	**Housing.com Verified Homes** (100+ checks); **NoBroker** owner verification; 99acres / MagicBricks tags	Mostly process and people; verify the poster, not each claim
Commute	**Zillow commute filter** (rush hour / off-peak); **Walk Score**, **TravelTime API**	Display commute to buyers; don't check broker claims
Deposit compliance	US fair-housing wording checkers in MLS tools	No Indian state-deposit checker found
Audit hash + PayPal anchor	None found	Unique to PropLock

Pitch framing: "Restb.ai proved photo compliance works for US MLSs; Zillow proved buyers want commute filters; we are the first to turn both into a verification gate for a payment."

Most market sources are company pages or press releases rather than independent evaluations; the NoBroker sources date from 2015–2016.

13. Build plan for the demo

Order	Work item	Effort	Demo impact
1	Fix `requirements.txt` (M8)	Minutes	Judges can run it
2	Mandate table + Check D (M1, M6 partial)	Low	Catches rent markup and bait listings
3	Claim extractor (M2)	Low	Catches lies in the title
4	Real PayPal mint with valid payload (M5)	Low	Real sandbox product, not a fake ID
5	UNVERIFIABLE + NEEDS_FIX / IN_REVIEW states (M3, D1)	Medium	Honest failure handling; fix-and-resubmit story
6	Pre-check endpoint	Low	"We help honest brokers" moment

7	Owner photos on listing + same-room check	Low	Closes the fake-photo hole
8	HMAC signed audit + `/verify` (M4)	Low	Real tamper-proof story
9	Store `report_json`, outbox (M7)	Medium	No orphans, no fabricated reports
10	Roles and auth (M6 full)	Medium	Closes the ground-truth hole

Roadmap slide (post-hackathon): listing expiry and versioning, full broker dashboard, live in-app capture page, reverse image search, Routes API with caching.

Appendix A. Probe results

Run against a scratch copy of `backend/` with a fresh database and no API keys.

```
P1 rent 55k vs owner 35k + Fully Furnished on Unfurnished flat: VERIFIED
P2 lie only in title/description, no structured claims:         VERIFIED  claims_evaluated=0
P3 honest 30-min claim to unknown destination:                  tolerance_exceeded=True  fabricated
actual=42
P4 destination 'a' fuzzy-matches:                               35  (tech park's time)
P5 PayPal description length:                                   283  (Catalog v1 limit 256)
P6 different broker photos → same audit hash:                   True
P7 self-registered ground truth → audit:                        VERIFIED
P8 zero photos at orchestrator level:                           VERIFIED
P9 rent=0, deposit=999999:                                      PASS  VERIFIED
```

Existing suite: `python -m pytest` → 9 passed. Add each probe as a regression test once fixed.

Appendix B. Caveats on legal figures

The legal figures in the current code and knowledge files should be verified against official gazette texts before the demo. The reviewer's understanding (not independently verified): the Karnataka Rent Act caps advance at 10 months (Section 11) rather than 2–3 months; the Model Tenancy Act 2021 caps residential deposits at 2 months; the 6-month Karnataka ceiling is a platform policy, not law. `tamil_nadu_tenancy_2019.md` contains a contradictory clause ("null, void, and enforceable"). Brokerage figures (1 month, 15 days) are market practice, not statute.

Appendix C. Sources

- PropLock AI Product Blueprint (PDF, 5 pages) and Feature 1 flow diagram
- Restb.ai Photo Compliance: https://www.restb.ai/solutions/photo-compliance/
- Heartland MLS and Restb.ai: https://kcrar.com/heartland-mls-and-restb-ai-announce-mls-partnership-for-computer-vision-photo-compliance
- Restb.ai, MLS Now: https://restb.ai/company/press-releases/mls-now/ · CVR MLS: https://restb.ai/company/press-releases/cvr-mls
- Google Patent US9990639: https://patents.google.com/patent/US9990639
- Onerent Scamlord AI: https://www.mannpublications.com/mannreportmanagement/2020/04/14/onerent-launches-scamlord-ai-fraud-detector/
- NYU fake-listing study (via UChicago): https://voices.uchicago.edu/201702busn3910001/?p=868
- Housing.com Verified Homes: https://housing.com/news/housing-verified-homes-our-assurance-of-trust/
- NoBroker (MediaNama, 2015): https://www.medianama.com/2015/02/223-nobroker-saif-partners-fulcrum-ventures/
- Business Today on NoBroker: https://www.businesstoday.in/magazine/features/taking-you-home/story/257816.html
- Zillow commute time filter: https://zillowgroup.com/news/zillows-commute-time-filter
- TravelTime case study: https://traveltime.com/case-study/realtor-commute-search-traveltime
- Walk Score profile: https://app.dealroom.co/companies/walk_score